

UNIFAAT

2026.2

06 - Revisão

Prof. Luan Lourenço

WSL: um Linux dentro do Windows

Virtualização roda um sistema operacional inteiro dentro de outro. Precisa de hypervisor ativo, e o hypervisor precisa de suporte de virtualização habilitado na BIOS — `VT-x` na Intel, `SVM Mode` na AMD.

PowerShell (Administrador)

instala o WSL 2 já com Ubuntu -- depois, reiniciar o PC

PS> wsl --install

confere a instalação e a versão

PS> wsl --list --verbose

Usuário e senha do Ubuntu são exclusivos do Linux — sem relação com o login do Windows.

Bit, byte e 256 combinações

O processador só distingue com segurança dois estados elétricos: passa corrente ou não passa. Cada estado vira um **bit** (0 ou 1). Um conjunto de 8 bits forma um **byte** — sinônimo de *octeto*.

● ● ● quantas combinações cabem em 8 bits?

$n = 2$ valores por bit, $p = 8$ bits $\rightarrow n$ elevado a p

$2^8 = 256$ combinações possíveis

contando do zero: de 0 até 255

O maior número com n bits é sempre $2^n - 1$ — com 8 bits, $255 =$ `11111111` .

A tabela de posições

Igual ao decimal (cada posição é uma potência de 10), o binário é posicional em base 2 — só que cada posição vale uma potência de 2, começando da direita.

● ● ● tabela de posições de um byte

posição	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
valor	128	64	32	16	8	4	2	1

Pra montar de cabeça: começa com 1 na posição mais à direita e vai dobrando a cada posição pra esquerda.

Binário → decimal

Alinha os dígitos pela direita, embaixo da tabela, e soma só as posições onde o dígito é 1.

● ● ● convertendo 101101

valor	32	16	8	4	2	1
dígito	1	0	1	1	0	1
# soma só onde o dígito é 1						
32 + 8 + 4 + 1 = 45						

Erro mais comum: alinhar pela esquerda. O último dígito da direita vale sempre 1, não importa o tamanho do número.

Decimal → binário: divisões sucessivas

Divide por 2, anota o resto (0 ou 1), repete com o quociente até chegar a zero — o binário é a leitura dos restos de baixo pra cima.

● ● ● convertendo 45

45 ÷ 2 = 22 resto 1

22 ÷ 2 = 11 resto 0

11 ÷ 2 = 5 resto 1

5 ÷ 2 = 2 resto 1

2 ÷ 2 = 1 resto 0

1 ÷ 2 = 0 resto 1

lendo os restos de baixo pra cima

101101

Mesmo número do slide anterior — confirma a conta. Erro mais comum: ler os restos de cima pra baixo, o que inverte o número.

Hexadecimal: encurtando a leitura do binário

Escrever binário longo é cansativo e propenso a erro. O hexadecimal (base 16) existe justamente pra encurtar essa leitura: como 16 é 2^4 , cada dígito hexadecimal representa exatamente 4 bits (um *nibble*) — um byte inteiro sempre cabe em só **2 dígitos hexadecimais**, em vez de 8 dígitos binários.

● ● ● convertendo o byte 01000001

nibbles: 0100 0001

hex: 4 1 -> 41

41 em hex = 65 em decimal = "A" na tabela ASCII

Hex no dia a dia: cor RGB

Uma cor em código, como `#FF0000` (vermelho puro), é só 3 bytes escritos em hexadecimal — um byte pra cada canal de cor, red, green e blue, cada um indo de 0 a 255.

● ● ● `#FF0000`

FF -> **11111111** -> **255** (canal R)

00 -> 00000000 -> 0 (canal G)

00 -> 00000000 -> 0 (canal B)

6 caracteres em hex, em vez de 24 dígitos binários — a mesma informação, só mais compacta e fiel, sem perder nada.

Binários: instalados nos dois lados

Todo comando de terminal é a busca por um arquivo binário com aquele nome — exceto `cd`, embutido no shell. Como o WSL é isolado, cada ferramenta precisa ser instalada **nos dois ambientes**.

```
# VS Code -- Windows (GUI)
instalador gráfico
# chamado de dentro do WSL:
code .
```

```
# Git -- Windows e WSL
Windows: git-scm.com
# WSL, via apt:
sudo apt install git -y
```

Comandos do terminal: o essencial

```
# navegar e listar  
pwd · ls · ls -la  
cd nome/ · cd .. · cd
```

```
# criar e manipular  
mkdir · mkdir -p  
touch · cat · cp · mv
```

! `rm` sozinho não apaga diretórios — dá erro de propósito. É preciso `rm -r` (recursivo), que apaga de dentro pra fora, sem lixeira de segurança.

Git local: a linha do tempo de commits

● ● ● ~/projeto

```
# cria o repositório .git nesta pasta
$ git init
# raio-x: o que mudou, o que é novo, o que já está pronto
$ git status
# separa o que vai entrar na próxima fotografia
$ git add .
# tira a fotografia, com uma mensagem explicando o que mudou
$ git commit -m "Cria anotacoes.txt..."
# mostra a linha do tempo inteira
$ git log
```

HEAD : soft x hard delete

HEAD aponta pro commit em que você está "parado" agora. HEAD~1 é um commit antes, sem precisar decorar nenhum hash.

~/projeto

```
# soft: desfaz o commit, mas mantém o trabalho pra recommitar
$ git reset --soft HEAD~1
# hard: desfaz o commit E descarta o trabalho, sem rastro
$ git reset --hard HEAD~1
```

A mesma dupla soft/hard delete reaparece em Banco de Dados: soft marca como removido, hard apaga a linha de vez.

HEAD é ponteiro, **remote** é config

~/projeto

```
$ cat .git/HEAD
ref: refs/heads/main
# cadastrando um remote:
$ git remote add origin git@github.com:usuario/repo.git
$ cat .git/config
[remote "origin"]
  url = git@github.com:usuario/repo.git
```

HEAD aponta pra outro arquivo, que guarda o hash. `git remote add` só escreveu duas linhas no **config** – nenhuma mágica de rede.

clone , **push** e **pull**

```
# baixar (1ª vez)  
git clone <url>
```

```
# enviar  
git push -u origin main
```

```
# receber  
git pull
```

clone é usado uma vez, quando o repositório local ainda não existe. **push** e **pull** são o dia a dia depois disso.

Chave SSH: pública x privada

● ● ● ~

```
$ ssh-keygen -t ed25519 -C "voce@email.com"
$ cat ~/.ssh/id_ed25519.pub
$ ssh -T git@github.com
Hi usuario! You've successfully authenticated...
```

A pública é o cadeado que vai pro GitHub, a privada nunca sai da máquina. Com isso, fecha a base da primeira parte do ano – a seguir, Python e orientação a objetos.

tarefa

TF 06: questões dissertativas

5 questões dissertativas cobrindo os três blocos de hoje — duas de conta binária, uma de navegação e filesystem, uma de binários instalados, e uma de Git.

Respondam com o raciocínio completo, não só o resultado final — nas contas de binário, o desenvolvimento vale tanto quanto a resposta.

Entrega pelo formulário abaixo — docs.google.com/forms/d/e/1FAIpQLScIz4IGfxjIz1oZdC6Y0Zb2bUa9gC8ztZUFPnRg8ewdWYeN0w/viewform